SOFTWARE TESTING – ASSIGNMENT 2

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (SwiftShip)

Course Outcome: CO2 – Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs and path testing.

Total Marks: 50

STUDENT DETAILS

Particulars	Details
Student Name	AATHISRI
Register Number	192521252
Course Code	CSA3712
Assignment	Assignment – 2

INTRODUCTION

SwiftShip is an online courier service that calculates parcel shipping charges and tracks each parcel until delivery. The system processes more than 40,000 bookings per day. A calculation error can create direct financial loss: undercharging reduces company revenue, while overcharging can lead to complaints, refunds and loss of customer trust. The purpose of this assignment is to design a compact but effective test suite using both black-box and white-box techniques, evaluate its coverage, and adapt it when new requirements are introduced.

The analysis follows the rules and pseudocode supplied in the assignment. Where the specification does not define an implementation detail—particularly the exact order in which the new fuel surcharge, express multiplier, discount and weekend fee are applied—the assumption is stated explicitly rather than treated as a confirmed business rule.

SYSTEM SPECIFICATION

Rule	Requirement
S1 – Weight	Weight must be greater than 0 kg and not more than 50 kg. Weight is entered up to one decimal place. Otherwise return INVALID.
S2 – Zone rate	Charge every started kilogram: Zone A = ₹40/kg, Zone B = ₹60/kg, Zone C = ₹90/kg. Any other zone is INVALID.
S3 – Express	Express delivery adds 50% to the charge. It is allowed only when weight is not more than 30 kg. Above 30 kg return NOT_ALLOWED.
S4 – Monthly discount	If the customer has booked 10 or more parcels in the current month, apply a 10% discount to the final charge.

PSEUDOCODE UNDER TEST – IMPORTANT OBSERVATION

```
1  if weight <= 0 or weight > 50 then return INVALID
2  rate = 40
3  if zone == "B" then rate = 60
4  else if zone == "C" then rate = 90
5  charge = 0; w = weight
6  while w > 0 do
7  charge = charge + rate
8  w = w - 1
9  if express then
10 if weight > 30 then return NOT_ALLOWED
11 charge = charge * 1.5
12 if parcels >= 10 then charge = charge * 0.9
13 return charge
```

The supplied pseudocode initializes the rate to ₹40 and changes it for zones B and C. It does not reject an unknown zone. Therefore, an invalid zone is incorrectly processed at the Zone A rate. The pseudocode also does not visibly validate the stated one-decimal-place input restriction. These are specification-to-code mismatches that the test suite must expose.

The loop adds one full zone rate for every started kilogram. For a positive weight with one decimal place, this is equivalent to ceil(weight) × rate. For example, 2.1 kg is charged as 3 kg.

TASK 1 – SELECT TEST CASE DESIGN STRATEGIES (5 MARKS)

Black-box testing derives tests from requirements and externally visible behaviour without examining the internal code. White-box testing derives tests from the program structure, such as statements, decisions, branches, loops and independent paths. The two approaches answer different questions: black-box testing asks whether the software implements the specification; white-box testing asks whether important parts of the implementation have been exercised.

SwiftShip component	Recommended strategy	Information required	Likely strengths	Limitations
Shipping-charge rules	Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), decision tables	S1–S4 specification and input/output rules	Finds invalid inputs, range-edge defects and interactions between express and discount rules	May miss unexecuted code paths or implementation-specific faults
shipCharge pseudocode	Statement, branch, condition and basis-path testing	Source pseudocode / control-flow graph	Finds missed branches, incorrect conditions, loop errors and dead or unexpected logic	Cannot prove that the implemented rule itself is the correct business rule
Parcel tracking	State-transition testing	State model and permitted transition rules	Finds illegal state changes, retry-limit errors and cancellation after pickup	Requires a complete state model; does not deeply test charge arithmetic

For the charge module, ECP is efficient because many values behave alike within a valid or invalid class. BVA is important because weight limits, the 30 kg express limit and the 10-parcel discount threshold are likely defect locations. A decision table is added because express and discount conditions can occur together. For the pseudocode, white-box branch and path testing checks the actual implementation. For tracking, state-transition testing is the natural choice because the validity of an action depends on the current state.

Combining these methods is more reliable than using only one. Black-box testing can expose the invalid-zone defect even if the relevant code executes normally; white-box testing can reveal an untested branch or loop behaviour; state-transition testing can detect invalid parcel lifecycle changes. The combined suite gives stronger evidence with less duplication.

TASK 2 – EVALUATE RANDOM TESTING (2 MARKS)

Random testing generates input combinations automatically from a selected input domain. For SwiftShip, a generator could choose weights from 0.0 to 50.0, zones from A/B/C and invalid zone strings, express as true or false, and parcel counts around and beyond the discount threshold. Each generated input is passed to shipCharge and its output is compared with an independently calculated expected result.

Random testing cannot replace systematic design. Important values such as 30.0, 30.1, 50.0, 50.1, 0.0, 0.1, 9 and 10 are rare if inputs are selected uniformly from a large domain. For illustration, if weights are selected uniformly from the 501 one-decimal values 0.0–50.0, the probability of selecting exactly 30.0 in one draw is 1/501 (about 0.20%); the probability of selecting either 30.0 or 30.1 is 2/501 (about 0.40%). The probability of hitting 50.0 is also 1/501. One thousand draws make hitting some values more likely, but do not guarantee every important boundary or combination.

A second difficulty is the test oracle: random input is useful only when the correct output is known. An independent reference calculation must apply validation, started-kilogram rounding, zone rate, express restriction and discount in the specified order. If the oracle repeats the same defect as the program, the test may

incorrectly pass. Random testing is therefore useful as an additional robustness technique, especially for unexpected combinations, but it should follow—not replace—ECP, BVA and decision-table tests.

TASK 3 – DESIGN AND EVALUATE EQUIVALENCE CLASSES (6 MARKS)

Equivalence Class Partitioning divides the input domain into groups expected to produce the same type of behaviour. One representative from each class is selected. The classes below are derived only from S1–S4.

Input	Valid equivalence classes	Invalid equivalence classes
Weight	0 < weight ≤ 50 kg; input has at most one decimal place	weight ≤ 0; weight > 50; more than one decimal place (format invalid under S1)
Zone	A, B or C	Any value other than A, B or C
Express	False; True with weight ≤ 30 kg	True with weight > 30 kg returns NOT_ALLOWED
Parcels	Integer count < 10; integer count ≥ 10	Negative or non-integer counts are not explicitly defined in S4; validate only if business rules specify them

For a compact representative suite, use one case to cover several valid classes when the conditions do not conflict, then isolate each invalid class so the cause of failure is clear. Expected charges use ceil(weight) × zone rate, then ×1.5 for permitted express, then ×0.9 when parcels ≥10.

ID	Weight	Zone	Express	Parcels	Expected result / purpose
E01	1.0	A	No	0	₹40 – valid A, standard, no discount
E02	2.1	B	No	10	₹180 × 0.9 = ₹162 – started kg, B, discount
E03	1.0	C	Yes	0	₹90 × 1.5 = ₹135 – C and express
E04	30.0	A	Yes	9	₹1,200 × 1.5 = ₹1,800 – express upper valid limit
E05	0.0	A	No	0	INVALID – lower invalid weight
E06	50.1	A	No	0	INVALID – upper invalid weight
E07	1.0	D	No	0	INVALID – invalid zone (pseudocode defect: returns ₹40)
E08	30.1	A	Yes	0	NOT_ALLOWED – express weight exceeds 30 kg
E09	1.00	A	No	0	Invalid input format if two decimal places

					are disallowed; pseudocode has no explicit format check

The E07 case is especially important: it is expected to return INVALID according to S2, but the supplied pseudocode defaults to rate 40 and is likely to return ₹40. This proves that the current implementation does not fully match the specification. E09 checks the one-decimal-place constraint; its execution depends on whether the application receives the original decimal text or only a numeric value. If formatting is discarded before this function, validation must occur at the input/API layer.

The class for parcels below 10 is represented by 0 and the class for 10 or more by 10. The specification does not define whether parcel count can be negative or fractional, so those cases should be clarified rather than assigned invented expected behaviour.

TASK 4 – DESIGN AND EVALUATE BOUNDARY VALUE TESTS (4 MARKS)

Boundary Value Analysis concentrates on values at and immediately around the edges of input ranges. For SwiftShip, the important boundaries are weight 0/0.1 kg, express limit 30/30.1 kg, maximum weight 50/50.1 kg, every started kilogram, and the monthly discount threshold 9/10 parcels.

BVA type	Typical selection	Approximate test count for n numeric inputs	Evaluation
Normal BVA	Minimum, minimum+step, nominal, maximum-step, maximum	4n+1	Efficient for valid-range boundary faults; does not test just-outside invalid values
Robust BVA	Normal BVA values plus just-below minimum and just-above maximum	6n+1	Covers invalid edges; stronger for input validation
Worst-case BVA	All combinations of selected boundary values	5^n for normal values; 7^n for robust values	Can grow rapidly and is expensive

For this module, robust BVA is recommended for the principal weight limits because invalid weights must be rejected. A focused, risk-based selection is more practical than a full worst-case combination of every input. The started-kilogram rule also needs values immediately below, at and above whole-kilogram points.

ID	Weight	Zone	Express	Parcels	Expected result
B01	0.0	A	No	0	INVALID
B02	0.1	A	No	0	₹40
B03	30.0	A	Yes	9	₹1,800
B04	30.1	A	Yes	9	NOT_ALLOWED
B05	50.0	C	No	0	₹4,500
B06	50.1	C	No	0	INVALID
B07	1.0	A	No	9	₹40

B08	1.0	A	No	10	₹36
B09	1.0	A	No	0	₹40 – exact whole kg
B10	1.1	A	No	0	₹80 – next started kilogram
B11	2.0	A	No	0	₹80
B12	2.1	A	No	0	₹120

The results show why a small set of carefully selected tests is more useful than simply generating many ordinary values. B03/B04 test the express threshold, B05/B06 test the maximum weight, B07/B08 test the discount threshold, and B09–B12 verify that every started kilogram is charged. Robust BVA gives the best balance between risk and execution effort.

TASK 5 – APPLY OTHER BLACK-BOX TECHNIQUES (3 MARKS)

ECP and BVA are good at individual input classes and boundaries, but they may miss defects caused by interactions. A decision table is suitable for the express and discount rules because both conditions can be true or false independently, subject to the express weight restriction.

Rule / condition	R1	R2	R3	R4	R5
Express selected?	No	No	Yes	Yes	Yes
Weight > 30 kg?	–	–	No	No	Yes
Parcels ≥ 10?	No	Yes	No	Yes	–
Expected action	Base charge	10% discount	50% express addition	Express then 10% discount	NOT_ALLOWED

ID	Weight	Zone	Express	Parcels	Expected result
D01	2.0	A	No	9	₹80
D02	2.0	A	No	10	₹72
D03	2.0	A	Yes	9	₹120
D04	2.0	A	Yes	10	₹108
D05	30.1	A	Yes	10	NOT_ALLOWED (express restriction takes precedence)

D04 checks the interaction between express and discount: ₹80 × 1.5 × 0.9 = ₹108. D05 checks that the express restriction is enforced even when the customer qualifies for a discount. The original ECP/BVA set contains some of these conditions, but the decision table makes the combinations explicit and ensures that both outcomes of each rule are tested.

TASK 6 – DESIGN AND EVALUATE STATE-TRANSITION TESTS (3 MARKS)

State-transition testing verifies that a parcel can move only through permitted states. The supplied model is:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED\n ↓\n FAILED_ATTEMPT\n ↓\n OUT_FOR_DELIVERY (up to two retries) OR RETURNED\nBOOKED → CANCELLED (cancellation allowed only before PICKED_UP)

Current state	Event	Next state	Validity
BOOKED	Pickup scan	PICKED_UP	Valid
BOOKED	Cancel booking	CANCELLED	Valid
PICKED_UP	Transit scan	IN_TRANSIT	Valid
IN_TRANSIT	Arrives for delivery	OUT_FOR_DELIVERY	Valid
OUT_FOR_DELIVERY	Successful delivery	DELIVERED	Valid
OUT_FOR_DELIVERY	Delivery unsuccessful	FAILED_ATTEMPT	Valid
FAILED_ATTEMPT	Retry, retry count within maximum	OUT_FOR_DELIVERY	Valid
FAILED_ATTEMPT	Retry limit exceeded	RETURNED	Valid
PICKED_UP	Cancel booking	CANCELLED	Invalid
DELIVERED	Return to transit	IN_TRANSIT	Invalid

Tests should cover every valid transition at least once, plus high-risk invalid transitions. An invalid cancellation after pickup could create inventory and billing inconsistencies. Reopening a delivered parcel could corrupt tracking history. The retry limit must be checked at the exact maximum: verify the first and second retries can return to OUT_FOR_DELIVERY, and that another failed attempt leads to RETURNED rather than an unlimited loop. The specification says “maximum two retries”; the exact event sequence should be confirmed with the product owner so the initial delivery attempt is not mistakenly counted as a retry.

TASK 7 – CONSTRUCT THE CONTROL FLOW GRAPH (4 MARKS)

A Control Flow Graph (CFG) represents the execution order of the pseudocode. Each node represents a statement or a group of straight-line statements, and each directed edge represents a possible transfer of control.

CFG node	Pseudocode lines	Meaning
N1	1–2	Validate weight; invalid path returns INVALID
N2	3–4	Initialize rate and test zone B
N3	4–5	Set rate to 60 for B
N4	6–7	Test zone C and set rate to 90 when true
N5	8–9	Initialize charge and loop variable
N6	10	Loop condition w > 0

N7	11–12	Add rate and decrement w
N8	13–15	Test express and reject weight > 30
N9	16	Apply express multiplier
N10	17–18	Test parcel count and apply discount
N11	19	Return charge

CFG (text representation)\nStart → N1 --valid--> N2\nN1 --invalid--> INVALID / End\nN2 --zone B--> N3 → N5\nN2 --not B--> N4 --zone C--> set rate 90 → N5\nN4 --not C--> N5 (rate remains 40)\nN5 → N6 --true--> N7 → N6\nN6 --false--> N8\nN8 --express false--> N10\nN8 --express true and weight > 30--> NOT_ALLOWED / End\nN8 --express true and weight ≤ 30--> N9 → N10\nN10 --parcels ≥ 10--> apply 0.9 → N11\nN10 --parcels < 10--> N11 → End

The compound condition at line 1 contains two atomic conditions: weight ≤ 0 and weight > 50, joined by OR. Tests should show each condition can independently make the overall decision true. The CFG also contains a loop at the started-kilogram calculation, an express decision, an express-weight decision and a discount decision.

The graph is derived from the supplied code, not from an ideal design. The code structure does not fully match the specification because an unrecognized zone follows the “not B, not C” route and retains the default Zone A rate. There is also no explicit decimal-place validation. The CFG helps reveal where to place tests, while the black-box specification tests prove the incorrect result.

TASK 8 – EVALUATE PATH TESTING AND CYCLOMATIC COMPLEXITY (4 MARKS)

Cyclomatic complexity estimates the number of linearly independent paths in a program. For a connected control-flow graph, it can be calculated as V(G) = E − N + 2P, where E is the number of edges, N is the number of nodes and P is the number of connected components (normally 1). It can also be calculated as the number of decision points plus 1.

For the supplied pseudocode, count the decision points as: (1) compound weight validation, (2) zone B, (3) zone C, (4) while loop, (5) express selected, (6) express weight >30, and (7) parcels ≥10. Therefore V(G) = 7 + 1 = 8. The compound condition at line 1 is one control-flow decision for cyclomatic complexity, although condition coverage must examine its two atomic conditions separately.

A basis-path set should exercise independent outcomes. Illustrative paths and data are shown below. Loop paths are represented with zero, one and multiple iterations; because valid weights are greater than zero, a zero-iteration loop is not reachable after successful weight validation. It is reachable only if the input is invalid and returns before the loop.

Path	Representative data	Path purpose / expected result
P1	weight=0.0, zone=A, express=No, parcels=0	Invalid weight early return
P2	weight=1.0, zone=A, express=No, parcels=0	Valid path; loop one iteration; ₹40
P3	weight=2.1, zone=B, express=No, parcels=0	Zone B; loop three iterations; ₹180
P4	weight=1.0, zone=C, express=No,	Zone C branch; ₹90

	parcels=0	
P5	weight=1.0, zone=A, express=Yes, parcels=0	Express true, allowed; ₹60
P6	weight=30.1, zone=A, express=Yes, parcels=0	Express true, disallowed; NOT_ALLOWED
P7	weight=1.0, zone=A, express=No, parcels=10	Discount branch; ₹36
P8	weight=1.0, zone=B, express=Yes, parcels=10	Combined zone, express and discount; ₹81

The listed cases are a practical basis-oriented set, not a claim that every possible loop iteration count is covered. A loop can execute many times for weights up to 50 kg, so test zero, one and multiple iterations where feasible and include the maximum valid weight. Some combinations are infeasible after early returns—for example, a weight above 50 cannot reach the zone, loop, express or discount statements. The loop is bounded for valid inputs, but its repeated addition must still be checked against the mathematical ceil(weight) rule.

The graph-based formula should produce the same value of 8 when the CFG nodes and edges are counted consistently. The decision-count method is shown explicitly here; a hand-drawn CFG can be counted using E − N + 2. Cyclomatic complexity gives a minimum basis-path count, not proof that the program is defect-free.

TASK 9 – JUDGE TEST ADEQUACY AND COVERAGE (3 MARKS)

Coverage is a measure of which parts of the program or requirements have been exercised. A single percentage is not sufficient to decide whether a financial calculation is safe. Different coverage criteria detect different classes of defects.

Coverage criterion	Meaning	Use for SwiftShip
Statement coverage	Every executable statement runs at least once	Basic minimum; may miss the false outcome of a decision
Branch coverage	Every decision outcome (true and false) is taken	Stronger; checks both sides of weight, zone, loop, express and discount decisions
Condition coverage	Each atomic Boolean condition is true and false at least once	Important for the compound weight check at line 1 and other Boolean conditions
Multiple-condition coverage	Relevant combinations of atomic conditions are exercised	Strongest of those listed for interacting Boolean conditions, but can increase test count

The Task 8 paths exercise the major statements and true/false outcomes of the main decisions when the full set is executed. To reach stronger condition coverage for the compound weight check, include one case where weight ≤ 0 is true while weight > 50 is false (for example 0.0), and another where weight ≤ 0 is false while weight > 50 is true (for example 50.1). A valid weight such as 1.0 makes both conditions false. This demonstrates each atomic condition independently.

For a money calculation used by 40,000 parcels each day, branch coverage plus targeted condition coverage is a reasonable release baseline, supplemented by ECP, BVA and decision-table tests. Multiple-condition coverage

may be applied to critical compound logic when the combinations are manageable. Even 100% statement or branch coverage would not automatically find the invalid-zone defect: the default Zone A path can be fully executed while still producing the wrong result for an invalid zone. A specification-based invalid-zone test is essential.

TASK 10 – COMPARE BLACK-BOX AND WHITE-BOX EFFECTIVENESS (4 MARKS)

Black-box tests are derived from S1–S4, while white-box tests are derived from the pseudocode and CFG. Their effectiveness should be judged by the defects they expose, not only by the number of test cases.

Defect / risk	Black-box contribution	White-box contribution
Invalid zone accepted as Zone A	Directly exposes it with zone D expected INVALID	Shows the default rate path is reachable when zone is neither B nor C
Wrong weight boundary handling	Tests 0, 0.1, 50 and 50.1 against S1	Exercises true and false outcomes of weight decision
Express >30 restriction missing or misplaced	Tests 30.0 and 30.1 with express enabled	Exercises express and weight sub-decisions
Discount threshold error	Tests 9 and 10 parcels	Exercises both outcomes of parcels decision
Started-kilogram loop error	Tests whole and fractional weights	Exercises loop zero/one/multiple iterations and repeated decrement
Incorrect tracking transition	Tests permitted and prohibited state changes	Not covered by charge-module CFG; state-transition testing is required

The invalid-zone defect is most clearly demonstrated by black-box expected-result comparison; white-box analysis helps explain why it occurs. White-box testing is especially effective at identifying untested decisions and loop paths. Black-box testing is stronger at checking whether the output agrees with the business rule, including behaviours that the code author may have misunderstood. Both methods overlap on weight, express and discount branches, so duplicate cases can be reduced if they exercise the same condition and provide no additional boundary or interaction evidence.

The same balance will not necessarily apply to other modules. A tracking workflow needs state-transition tests, while a complex numeric algorithm may need stronger path and data-flow testing. Test strategy must be selected according to the risks and structure of each component.

TASK 11 – RECOMMEND AND DEFEND THE FINAL TEST SUITE (4 MARKS)

The recommended release suite is a compact risk-based combination rather than the largest possible collection. Retain tests that uniquely cover a specification class, boundary, rule interaction, program branch or state transition. Remove only exact duplicates that add no new coverage.

Test group	Included coverage	Technique	Suggested count
Core input validity	Invalid/valid weight, valid and invalid zone, express limit, parcel threshold	ECP + BVA	10

Started-kilogram calculation	Whole and fractional weights, including multiple loop iterations	BVA + loop testing	3
Rule interactions	Express on/off with discount on/off; prohibited express weight	Decision table	5
White-box paths	Weight rejection, zone branches, loop, express branches and discount branch	Branch / basis-path testing	8 (overlap with cases above where possible)
Tracking	Valid lifecycle, cancellation, two retries, invalid cancellation and invalid post-delivery transition	State-transition testing	10

The counts are planning targets, not an instruction to duplicate identical inputs. The final execution list should merge overlapping cases while preserving the distinct assertions. The core charge suite can be automated and run on every build; the tracking tests can be run as a separate workflow suite. A reasonable release target is branch coverage for the charge module, targeted condition coverage for compound conditions, all valid equivalence classes, critical boundaries, and all defined tracking transitions.

This suite is preferable to random-only testing because every important boundary and rule combination is deliberately represented. It is preferable to exhaustive worst-case BVA because the latter can create many combinations without proportionate risk reduction. Remaining risks include unspecified handling of negative/fractional parcel counts, rounding/currency precision, invalid decimal text, and the exact definition of retry counting. These should be clarified with the business owner.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE TEST SUITE (8 MARKS)

Changed requirement

SwiftShip adds an 8% fuel surcharge for Zone C parcels and a weekend delivery option that adds ₹50 per parcel. The assignment states that two new decisions are inserted after line 16. The suite must still complete within 30 minutes before every release. The appropriate decision is MODIFY the existing suite: the original requirements remain valid, but new classes, combinations, expected results and code paths must be added.

A. Impact analysis (2 marks)

The specification now has two additional conditions: (1) zone C triggers an 8% fuel surcharge; (2) weekend delivery adds ₹50. Zone A and B should not receive the fuel surcharge under the stated rule. Weekend fee applies when weekend delivery is selected. Existing weight, zone, express and discount boundaries remain relevant.

The equivalence classes must be extended to include Zone C versus non-C for surcharge, and weekend selected versus not selected. The decision table must include the combinations of Zone C/non-C and weekend/weekday, alongside express and discount where practical. New boundaries are not numeric thresholds in the supplied wording, but the fee calculation must be tested at zero and one weekend parcel if the fee is per parcel. Since the interface describes a weekend delivery option, tests must check both selected and not selected.

The two added decisions increase the number of decision points from 7 to 9. Therefore, under the same simple-decision assumption, cyclomatic complexity changes from V(G)=8 to V(G)=10. The exact value must be confirmed against the revised source code and its CFG; if either new condition is implemented with compound

Boolean logic, the decision count may differ. The 30-minute limit matters because a regression suite must be repeatable on every release. Test prioritisation, automation and removal of duplicate cases are necessary.

B. Evaluation of the original suite (2 marks)

Most original tests remain valid for S1–S4, but their expected results may change when the new surcharge or weekend option is enabled. Tests with Zone C must be reviewed because the new 8% surcharge changes the amount. Tests for Zones A and B remain unchanged for fuel surcharge, unless weekend delivery is selected. Existing express and discount tests remain necessary, but combined cases must be recalculated under the confirmed order of operations.

The original suite does not cover the new fuel-surcharge decision or the weekend-fee decision. It therefore cannot achieve full branch coverage of the revised code. New paths must exercise Zone C and non-C outcomes for the surcharge decision, and weekend selected/not selected outcomes for the weekend decision. The updated suite should target branch coverage of the revised code plus the original critical boundaries and decision-table combinations.

C. Adaptation or redesign (2 marks)

Modify the suite by retaining the original ECP/BVA/state-transition tests and adding a small number of high-value cases for the new features. The following calculations use an explicit provisional assumption: percentage adjustments are applied to the base shipping charge, express multiplier and fuel surcharge are applied before the 10% discount, and the fixed weekend fee is added after percentage adjustments. The assignment does not state the exact order; the product owner must confirm it before these values become authoritative.

ID	Weight	Zone	Express	Parcels	Weekend	Expected calculation under stated assumption
N01	1.0	A	No	0	No	₹40 – no surcharge or weekend fee
N02	1.0	C	No	0	No	₹90 × 1.08 = ₹97.20
N03	1.0	B	No	0	Yes	₹60 + ₹50 = ₹110
N04	1.0	C	No	10	No	₹90 × 1.08 × 0.9 = ₹87.48
N05	2.0	C	Yes	0	Yes	₹180 × 1.08 × 1.5 + ₹50 = ₹341.60
N06	30.1	C	Yes	10	Yes	NOT_ALLOWED – express restriction applies before charge adjustments

N01 and N02 distinguish the fuel-surcharge decision. N03 checks the fixed weekend fee for a non-C zone. N04 combines Zone C surcharge with the monthly discount. N05 combines Zone C surcharge, express and weekend delivery. N06 confirms that the express weight restriction is still enforced when new options are selected. The updated test set should also include Zone C with weekend not selected and Zone A/B with weekend selected, so both outcomes of each new decision are executed.

For the 30-minute limit, automate deterministic calculation tests and run them as a focused regression suite. Prioritise changed code and its dependencies first, then run the complete critical regression set. Use a test data file with expected outputs generated independently from the implementation. Record execution time and coverage for each release. If the complete selected suite exceeds 30 minutes, remove duplicate tests only after confirming their unique coverage is retained; do not remove critical boundary or invalid-input tests.

D. Evidence-based defence (2 marks)

The modified suite is supported by the following evidence: ECP covers every valid and invalid input class from S1–S4 and the new option classes; robust BVA covers 0, 0.1, 30, 30.1, 50 and 50.1 kg plus the 9/10 parcel threshold and started-kilogram changes; the decision table covers express/discount interactions and is extended for Zone C surcharge and weekend delivery; basis-path tests target the revised CFG and the new decisions; state-transition tests continue to cover parcel tracking.

The estimated cyclomatic complexity rises from 8 to 10 if the two added conditions each create one decision. At least 10 independent paths are therefore a useful structural planning baseline, but the final test count may be lower than the sum of all groups because one test can cover more than one requirement or branch. The suite should report statement, branch and targeted condition coverage, not a single unexplained percentage. Its execution must be measured and kept within 30 minutes through automation and risk-based selection.

Final decision: MODIFY the original suite. Retaining it unchanged would leave the two new decisions untested. Completely redesigning it would waste useful tests that still cover the original requirements. A modified, traceable regression suite gives the best balance of defect detection, execution time and maintenance effort.

ADDITIONAL ELABORATION – TASK 1

Strategy selection should begin with risk identification. The charge calculator has a high financial impact because even a small error is repeated across thousands of parcels. Therefore, tests should not be selected only because they are easy to execute; they should target the requirements most likely to cause incorrect billing.

Black-box testing is particularly valuable when the tester has access to the requirement document but not the internal implementation. It can reveal missing validation, incorrect rounding, wrong rates and incorrect final amounts. White-box testing complements it by showing whether each branch and loop outcome has actually been executed. Neither approach alone is sufficient: code coverage does not prove that the expected business result is correct, and specification tests may not reveal an unexecuted internal branch.

For tracking, state-transition testing should be supported by transition coverage and selected transition-pair tests. A transition test checks one permitted move, while a transition-pair test checks whether two consecutive moves are valid. This is useful for confirming that a parcel cannot skip directly from BOOKED to DELIVERED or be cancelled after pickup.

ADDITIONAL ELABORATION – TASK 2

Random testing is most useful after the systematic test cases have been established. A random generator can explore combinations that a tester may not think of, such as unusual valid fractional weights combined with different zones and parcel counts. A fixed random seed should be recorded so that a failure can be reproduced.

Random testing should be constrained by the specification. The generator should deliberately include both valid and invalid zones, values around the permitted weight range, and express requests above and below the 30 kg limit. Purely uniform generation may spend too much time on ordinary values and too little time on rare but important boundaries. A hybrid approach—systematic boundary cases plus random supplementary cases—provides better defect detection.

ADDITIONAL ELABORATION – TASK 3

Each equivalence class should be independent enough that a failure can be diagnosed. For example, an invalid weight test should use a valid zone and ordinary express/parcel values; otherwise, if the result is INVALID, the tester may not know which input caused the rejection. The same principle applies to invalid-zone testing: use a valid weight and otherwise valid options.

Valid classes should also be combined carefully to reduce the number of cases without hiding the cause of failure. A test using Zone C, express delivery and a qualifying parcel count can cover several valid classes, but separate tests are still needed to isolate each decision and to confirm the calculation order. The smallest possible test set is not automatically the best set if it makes failures difficult to diagnose.

ADDITIONAL ELABORATION – TASK 4

Boundary tests should include values immediately below and above the whole-kilogram points because the started-kilogram rule changes the charge at those points. For example, 2.0 kg is charged as two kilograms, while 2.1 kg is charged as three kilograms. This is a business rule rather than a simple input-validity limit, so it deserves dedicated tests.

Robust BVA is recommended over normal BVA because invalid values must be rejected, and over full worst-case BVA because the number of combinations grows quickly. The practical approach is to test each high-risk boundary independently and then add a few selected combinations where two boundaries may interact, such as 30.1 kg with express selected and 50.1 kg with a valid zone.

ADDITIONAL ELABORATION – TASK 5

A decision table is valuable because it makes the expected result explicit for every meaningful combination of conditions. It also helps reviewers detect missing rules. Each column in the table should become at least one executable test case, and each test should identify the expected action—not merely say that the program should work.

Where conditions are mutually exclusive, the table should show that clearly. For example, when express is selected and weight is above 30 kg, the expected result is NOT_ALLOWED; the discount should not convert that rejected request into a charge. This precedence should be checked against the supplied specification and confirmed if implementation details are unclear.

ADDITIONAL ELABORATION – TASK 6

State-transition tests should record the initial state, triggering event, expected next state and retry count. This makes the test repeatable and provides evidence if the application enters an illegal state. The retry count should be checked at zero retries, one retry, the maximum of two retries and the next attempted retry.

Invalid transitions are selected according to business risk. Cancellation after pickup may cause a parcel to disappear from operational tracking while it is already in the delivery network. A transition out of DELIVERED may overwrite a completed delivery record. These are high-priority negative tests because they can affect customer trust, operational accountability and billing.

ADDITIONAL ELABORATION – TASK 7

When reviewing the control-flow graph, every decision must have a clearly labelled true and false edge. The while loop has an edge back to its condition after the charge increment and weight decrement. Its false edge exits the loop and continues to the express decision. The early return edges for invalid weight and disallowed express delivery terminate execution immediately.

The CFG is also a review tool for requirement traceability. The absence of a decision that checks whether zone is A, B or C is visible in the graph: the code only tests B and C after initializing the default rate. Therefore, an

unknown zone follows the same route as Zone A. This is a clear example of how a structural view helps explain a defect first exposed by a black-box test.

ADDITIONAL ELABORATION – TASK 8

Cyclomatic complexity is a measure of independent control-flow paths, not the total number of possible executions. A loop can repeat a variable number of times, so the total number of execution traces may be much larger than the basis-path count. Basis-path testing selects a manageable set of independent paths, while loop testing separately checks zero, one and multiple iterations where those cases are feasible.

For this program, a valid weight is always greater than zero, so the loop should execute at least once after weight validation succeeds. A zero-iteration loop is therefore not reachable on a valid input. It is still useful to reason about loop exit behaviour and to test the early-return route for invalid weight. The maximum valid weight of 50 kg gives a useful upper loop-count test.

ADDITIONAL ELABORATION – TASK 9

Statement coverage can be misleading because a statement may execute even when one of its decision outcomes has never been tested. Branch coverage is stronger because it requires both outcomes of each decision. Condition coverage adds evidence that each atomic Boolean expression can independently evaluate to true and false.

For financial software, coverage should be interpreted together with requirement coverage and data assertions. A test may execute every statement and still return the wrong amount. The invalid-zone example proves this point: the program may execute the default rate calculation and achieve high structural coverage, but only a specification-based assertion that an unknown zone must return INVALID reveals the defect.

ADDITIONAL ELABORATION – TASK 10

The comparison should be recorded in a defect-to-test traceability matrix. For each defect, record the test ID, expected result, actual result, technique and whether the defect was found by black-box testing, white-box testing or both. This creates evidence for the evaluation instead of relying on a general claim that one technique is better.

Duplicate tests can be removed only when they exercise the same relevant input class, boundary, decision outcome and expected behaviour. Two tests with different values may still be duplicates for branch coverage, but one may be essential for boundary coverage. Therefore, reduction should be based on coverage contribution and defect-detection purpose rather than identical or similar input values alone.

ADDITIONAL ELABORATION – TASK 11

The regression suite should be maintained as a traceable collection of test cases. Each case should have a unique ID, requirement reference, input data, expected result, actual result, pass/fail status and execution date. Automated calculation tests are suitable for frequent execution because their inputs and expected results are deterministic.

Test prioritisation should place invalid inputs, financial boundaries, express restrictions, discount calculations and recently changed code near the beginning of the run. Tracking tests should be included in the release regression plan because incorrect state changes can create operational failures even when the charge amount is correct. The final report should state the actual run time rather than assuming that the 30-minute target has been met.

ADDITIONAL ELABORATION – TASK 12

Impact analysis must distinguish unchanged requirements from changed behaviour. The original weight limits, zone rates, express limit and discount threshold remain in force. The new surcharge changes expected results for Zone C, while the weekend option adds a fixed fee when selected. Existing Zone C expected values must therefore be recalculated for the new rule, and new cases must exercise both outcomes of each new decision.

Before finalising the new expected amounts, the business owner must confirm the order of operations. For example, applying a 10% discount before or after a fuel surcharge can produce different totals. The assignment gives the percentage and fixed fee but does not explicitly state the sequence. The report therefore labels its calculation order as a provisional assumption. This is an important testing clarification, not a detail that should be silently guessed.

To remain within 30 minutes, the team should automate the deterministic charge tests, reuse common test data, remove only proven duplicates and execute high-risk tests first. A short smoke suite can run after every build, while the full modified regression suite runs before release. Execution duration and coverage should be recorded for each release so that the time limit is supported by evidence.

CONCLUSION

The SwiftShip assignment demonstrates that effective testing requires more than generating a large number of inputs. ECP and BVA provide systematic input coverage, decision tables expose interactions, state-transition testing protects the tracking lifecycle, and white-box CFG and path testing provide evidence about the implementation structure. Coverage measurements support the release decision but cannot replace checking expected results against the specification. The Day-4 changes require the suite to be modified with new tests for the Zone C fuel surcharge and weekend delivery option. A compact, automated, risk-based regression suite is the most suitable approach for a system processing more than 40,000 parcels per day.

REFERENCES

1. SwiftShip Software Testing Assignment 2 – supplied assignment specification and mark distribution (14 pages).

2. Course Outcome CO2 topics stated in the assignment: black-box and white-box testing, ECP, BVA, decision tables, state-transition testing, CFG, path testing and coverage.

3. Expected results and calculations in this report are derived from the supplied S1–S4 rules. Day-4 calculations are provisional where the supplied assignment does not specify adjustment order.